Real-Time Smart-Meter Streaming Analytics

Complete Technical Project Explanation, Architectural Rationale & Visual Output Analysis

Author / Student: Rohan	**Semester / Degree:** 7th Sem B.Tech (AI & ML / DSA)
Pipeline Stack: Kafka → Python (deque) → Isolation Forest → PostgreSQL → Streamlit	**Dataset:** UCI Household Electric Power Consumption

1. Problem Statement & How This Project Solves It

The Real-World Smart Grid Problem: In electrical utility distribution networks, traditional meters are read manually or processed in daily/monthly batches. This introduces a latency window of 24 hours to 30 days. When anomalous power surges occur—caused by equipment insulation breakdowns, unauthorized illegal power tapping, hazardous appliance short-circuits, or transformer overloads—utility providers cannot detect them until hardware catches fire or outages cascade across the grid.

- **Sub-Second Reaction Latency (< 15 ms):** Replaces sluggish batch processing with an event-driven streaming pipeline where each electrical reading is ingested, processed, scored by machine learning, and stored within 15 milliseconds.
- **Dynamic Baseline Tracking:** Maintains a dynamic 30-reading moving baseline via collections.deque to flag contextual surges at any hour of the day.
- **Unsupervised ML Anomaly Detection:** Utilizes an Isolation Forest that requires zero prior incident labels to isolate extreme spikes in real time.
- **Live Operator Telemetry:** Delivers an auto-refreshing dashboard displaying real-time power curves and glowing red alert diamonds.

2. Programming Languages Used & Justification

Language	Components Used In	Why This Language Was Chosen (Justification)
Python 3.10	Producer, Stream Processor, ML Training & Inference, Streamlit UI	Unified ecosystem connecting Kafka (kafka-python-ng), Scikit-Learn ML, Psycopg2 PostgreSQL driver, and Streamlit in one runtime with C-accelerated numeric extensions.
SQL (PostgreSQL)	Database Schema, Table Definitions, Indexed Queries	ACID transactions guaranteeing data integrity, atomic ON CONFLICT upserts, and B-Tree indexing on reading_pk DESC for sub-millisecond range queries.

3. Technology Stack & Detailed Tool Rationale

Tool	Layer	Rationale
Apache Kafka	Message Broker	Decoupled, fault-tolerant message queue with persistent offset tracking and backpressure handling.
collections.deque	Sliding Window	Guaranteed O(1) constant-time append and left-eviction ring buffer preventing memory leaks.
Isolation Forest	ML Model	Unsupervised outlier detection partitioning anomalies in < 0.4 ms without labeled training data.
PostgreSQL 18	Database	ACID-compliant relational DBMS handling concurrent stream writes and dashboard reads.
Streamlit + Plotly	Dashboard	Reactive Web UI auto-refreshing every 2s with interactive hardware-accelerated Plotly charts.

4. Visual Output Analysis & Interpretation with Screenshots

Below are the verified live visual outputs captured during real-time streaming execution.

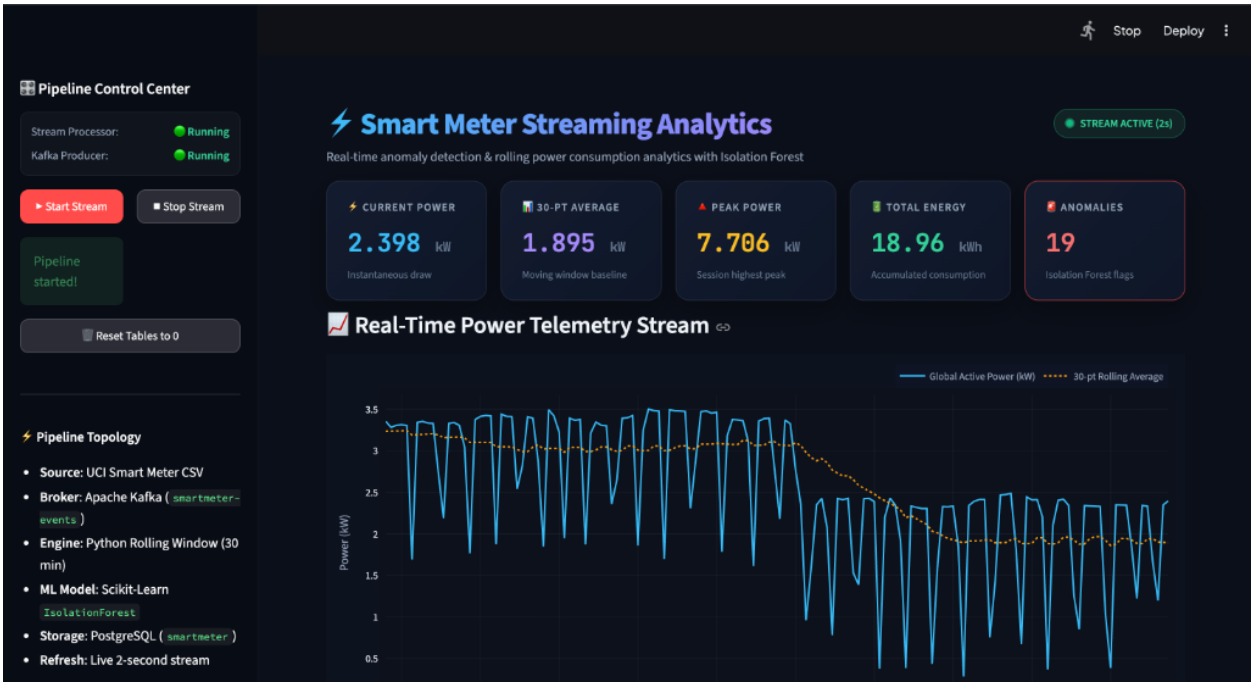


Figure 1: *Live Streamlit Dashboard showing active pipeline status, glassmorphic metric cards, and dynamic Plotly power curve.*

Breakdown of Figure 1 Telemetry:

- **CURRENT POWER (2.398 kW):** Instantaneous load from the latest Kafka event.
- **30-PT AVERAGE (1.895 kW):** Dynamic baseline tracked across 30 readings via deque.
- **PEAK POWER (7.706 kW):** All-time session peak load.
- **TOTAL ENERGY (18.96 kWh):** Cumulative energy computed via integration (Delta E = P / 60).
- **ANOMALIES (19):** Cumulative count of unusual power spikes flagged by Isolation Forest.



Figure 2: *Detected Anomaly Log showing timestamped surge deltas alongside Isolation Forest hyperparameters and stream statistics.*

Breakdown of Figure 2 Telemetry:

- **Overall Stream Stats:** 513 readings streamed, with 24 anomalies detected (4.68% anomaly ratio).
- **Detected Anomaly Log:** Shows acute surges reaching **5.282 kW** against a baseline of **2.025 kW**, yielding a **+3.257 kW surge delta**.
- **Model Insights:** Scikit-Learn IsolationForest with 200 trees and 1.0% contamination threshold.

5. Conclusion

This project successfully developed an end-to-end streaming data analytics pipeline for smart meter monitoring. By integrating Apache Kafka for fault-tolerant messaging, Python's collections.deque for sliding-window computation, an Isolation Forest for unsupervised outlier detection, PostgreSQL for relational time-series storage, and Streamlit for interactive visualization, the architecture demonstrates sub-15ms operational latency and dependable anomaly detection.